

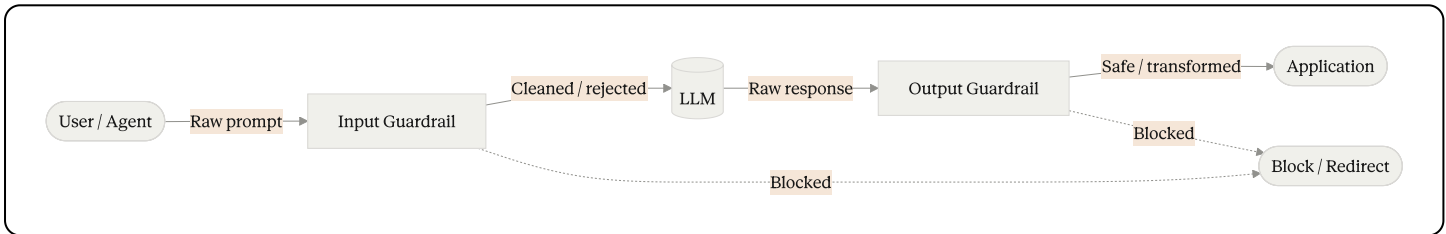
LLM Guardrails: The Complete 2026 Guide

What it is, how it works, why it matters, and which framework fits which use case — covering every major open-source and enterprise guardrail system.

1. What Are LLM Guardrails?

LLM Guardrails are **runtime safety and control layers** that sit between your application and the LLM. Unlike training-time alignment (RLHF, Constitutional AI), guardrails operate **after** the model is frozen — at inference time — intercepting, filtering, validating, or redirecting both inputs (user prompts) and outputs (model responses) before they reach end users or downstream systems.

Think of them as a **multi-layer firewall purpose-built for generative AI**:

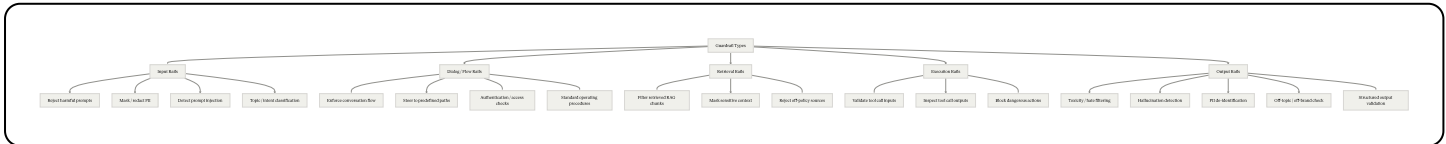


Why guardrails cannot be skipped in 2026

- **The EU AI Act** is now in force — regulated industries (finance, healthcare, government) must demonstrate documented runtime controls, not just model alignment.
- **The OWASP LLM Top-10 (2025)** lists Prompt Injection as #1 vulnerability. Research (PoisonedRAG, USENIX Security 2025) showed just 5 crafted documents can manipulate RAG responses 90% of the time.
- **Agentic blast radius**: in 2023, a guardrail failure meant a bad chat response. In 2026, it means a bad **action** — money transferred, database deleted, email sent to wrong party.
- **14 new AI-specific attack techniques** were added to the MITRE ATLAS framework in 2025 alone.
- **LLM-judge-based safety classifiers** add 5-11 seconds per check; purpose-built guard models run in 29ms — you *must* use dedicated tools at production scale.

2. The Guardrail Taxonomy

There are five architecturally distinct types of guardrail:



Implementation approaches (speed vs. power trade-off)

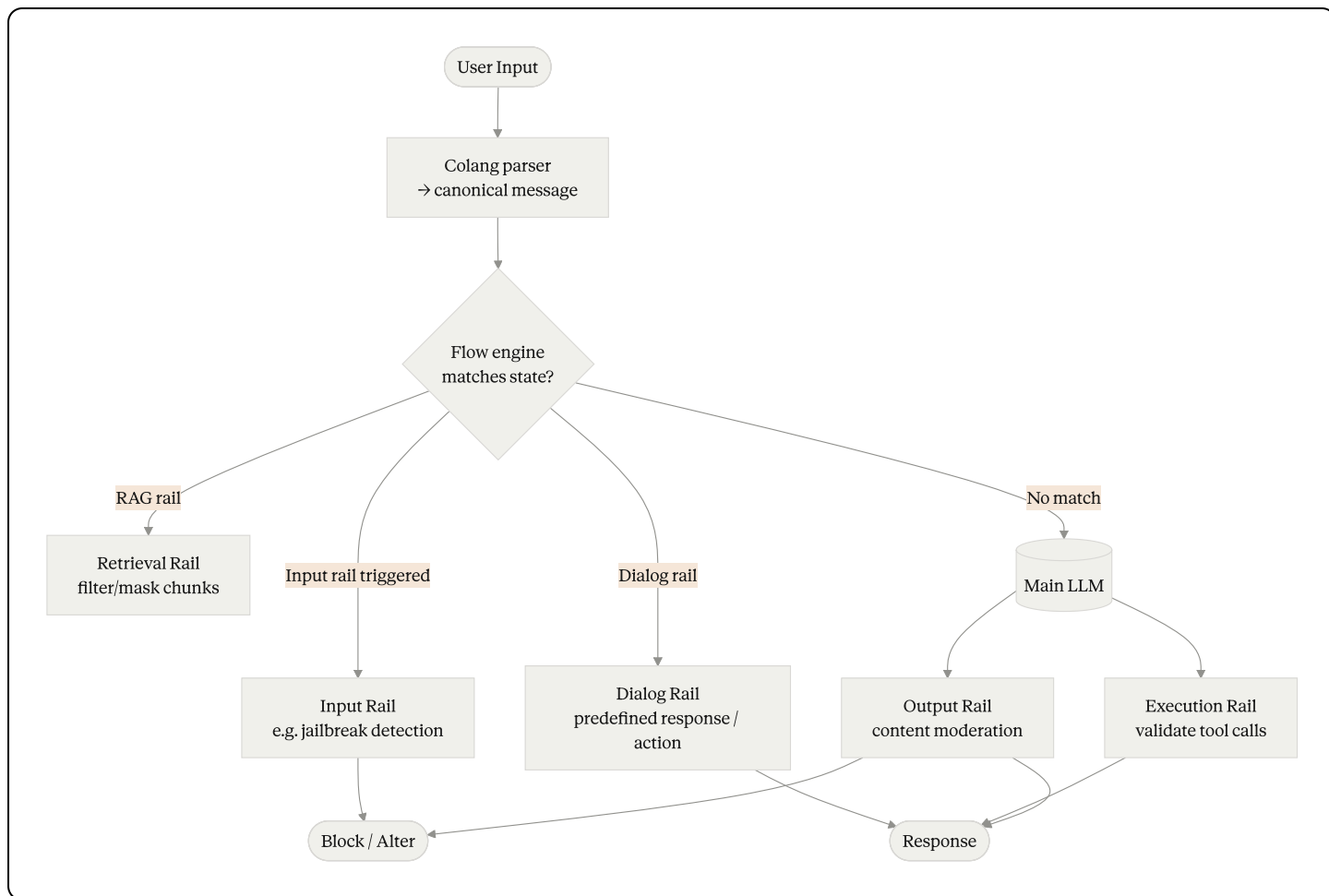
Approach	Latency	Robustness	Cost	Use when
Regex / keyword rules	<1ms	Low (bypassed by paraphrasing)	Free	High-volume first-pass filter for known patterns
Classical ML classifier	5-50ms	Medium	Low	Stable categories with labeled training data
Purpose-built guard model	10-100ms	High	Low-medium	Production real-time protection (LlamaGuard, ShieldGemma)
LLM-as-judge	500ms-11s	High (but jailbreakable itself)	High	Nuanced / novel policies; async post-hoc auditing
Constrained generation	+0ms on output	Highest (structural)	Low	When output schema must be guaranteed

3. The Frameworks, In Depth

3.1 NeMo Guardrails (NVIDIA)

What it is: The most comprehensive open-source LLM guardrail toolkit, from NVIDIA. Released April 2023 (arXiv 2310.10501), now on the Thoughtworks Technology Radar at **Adopt** (Nov 2025). Used across teams from simple chatbots to complex agentic workflows.

How it works: NeMo Guardrails uses **Colang** — a purpose-built domain-specific language — to define a dialogue state machine. Conversations are represented as *canonical form* messages; Colang flows control what happens at each state (reject, redirect, call action, invoke LLM). The engine is built on a **runtime inspired by dialogue management systems**, making rails independent of the underlying LLM and interpretable.



Five rail types (GitHub: NVIDIA-NeMo/Guardrails):

1. **Input rails** — applied to user input before any LLM call; can reject or alter (mask PII, rephrase)
2. **Dialog rails** — operate on canonical messages; determine flow: predefined response, LLM call, or action
3. **Retrieval rails** — filter/mask RAG chunks before they reach the LLM context
4. **Execution rails** — validate inputs/outputs of custom actions / tool calls (a.k.a. function calling)
5. **Output rails** — applied to final LLM response; can reject (return canned response) or alter

Key 2025-2026 updates (from releases page):

- **IORails:** new optimized Input/Output engine with parallel execution of NemoGuard microservices (content-safety, topic-safety, jailbreak-detection), unique request IDs, async `check_async` method
- **LangGraph + tool calling integration** for safe multi-agent workflows
- **OpenAI-compatible server** including `/v1/models` endpoint
- **GuardrailsMiddleware** for seamless LangChain agent integration
- **Community integrations:** PolicyAI, CrowdStrike AIDR, Trend Micro AI Guard, Prompt Security, AutoAlign, Patronus Lynx
- **Colang 2.0:** cleaner syntax, better multi-turn support
- **Streaming support** through output rails for improved performance

- **Three NVIDIA microservices:** content-safety, topic-control, jailbreak-detection (GPU-accelerated, sub-50ms)
- **NemoGuard 8B** model: F1 0.793 on OpenAI Moderation, 0.875 on HarmBench — respectable but not SOTA

Performance: GPU-accelerated, sub-50ms latency for classifier rails; complex dialog flows 20-50ms overhead. Dialog validation can be heavier.

Strengths: Most comprehensive OSS conversational control, full dialog management, multi-LLM orchestration, active NVIDIA backing, broad integration ecosystem, interpretable Colang policies.

Weaknesses: Colang learning curve, NemoGuard 8B is not the strongest guard model by benchmark, smaller community than LLM Guard, requires engineering investment to define policies.

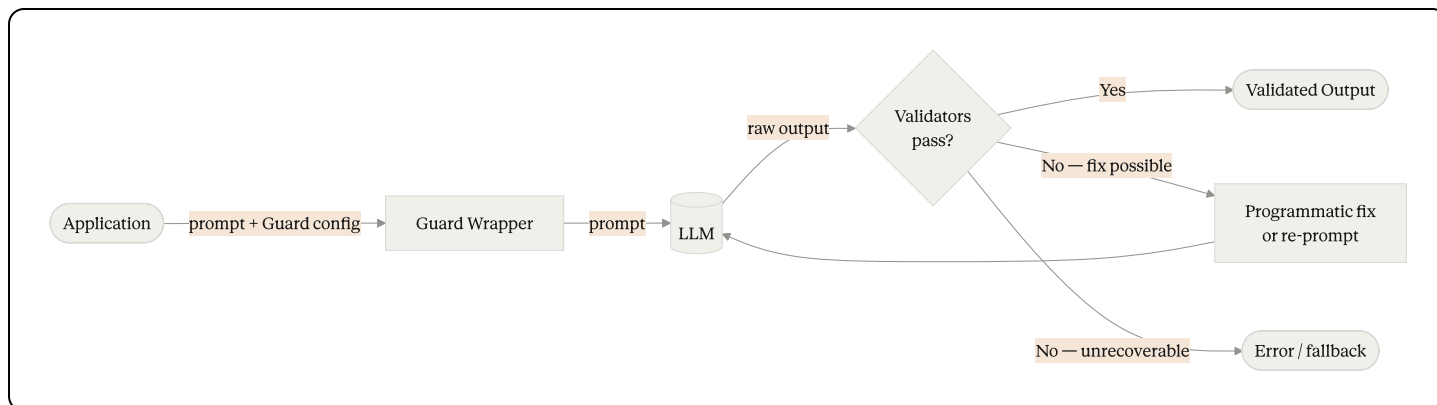
Best for: Teams wanting **full programmatic control over conversational flows**, policy-as-code, GPU-accelerated classification, and are comfortable in the NVIDIA/Colang ecosystem. Especially strong for regulated enterprise chatbots and multi-step agent pipelines.

License	Apache 2.0
GitHub	github.com/NVIDIA-NeMo/Guardrails
Stars	~4.5k
Language	Python
Install	<code>pip install nemoguardrails</code>

3.2 Guardrails AI

What it is: Open-source framework for **structured output validation and programmatic output enforcement**. Uses a RAIL (Reliable AI Language) spec to declare what your LLM outputs must look like, then validates and (optionally) corrects deviations.

How it works: You define a Guard object that wraps your LLM call. Guardrails AI wraps the LLM call, validates the output against your declared validators, and if validation fails, either retries with reprompt instructions, fixes the output programmatically, or returns an error. The **Guardrails Hub** provides 60+ pre-built validators contributed by the community.



Validator categories (60+ in Hub):

- **Type validators:** must be integer, float, date, URL, email, JSON
- **Content validators:** no profanity, no PII, toxicity score, sentiment bounds
- **Format validators:** regex match, length bounds, enum membership
- **Semantic validators:** LLM-judge validators for truthfulness, relevance, bias
- **Custom validators:** write any Python function as a validator

Key features:

- **RAIL spec:** XML or Pydantic-like declaration of output schema + validator chain
- **Guardrails Hub:** community marketplace for pre-built validators
- **Python + JavaScript SDKs**
- **Re-ask loop:** automatic retry with failure context injected into reprompt
- **Streaming support** with partial validation
- **Enterprise version:** team collaboration, managed deployment, priority support

Strengths: Best OSS tool for **structured output guarantees**; composable validator pipelines; large community Hub; works with any LLM; Python + JS.

Weaknesses: Adds retry latency when validation fails; RAIL spec has a learning curve; less suited for conversational flow control (NeMo Guardrails is stronger there); less focused on adversarial prompt injection than dedicated security tools.

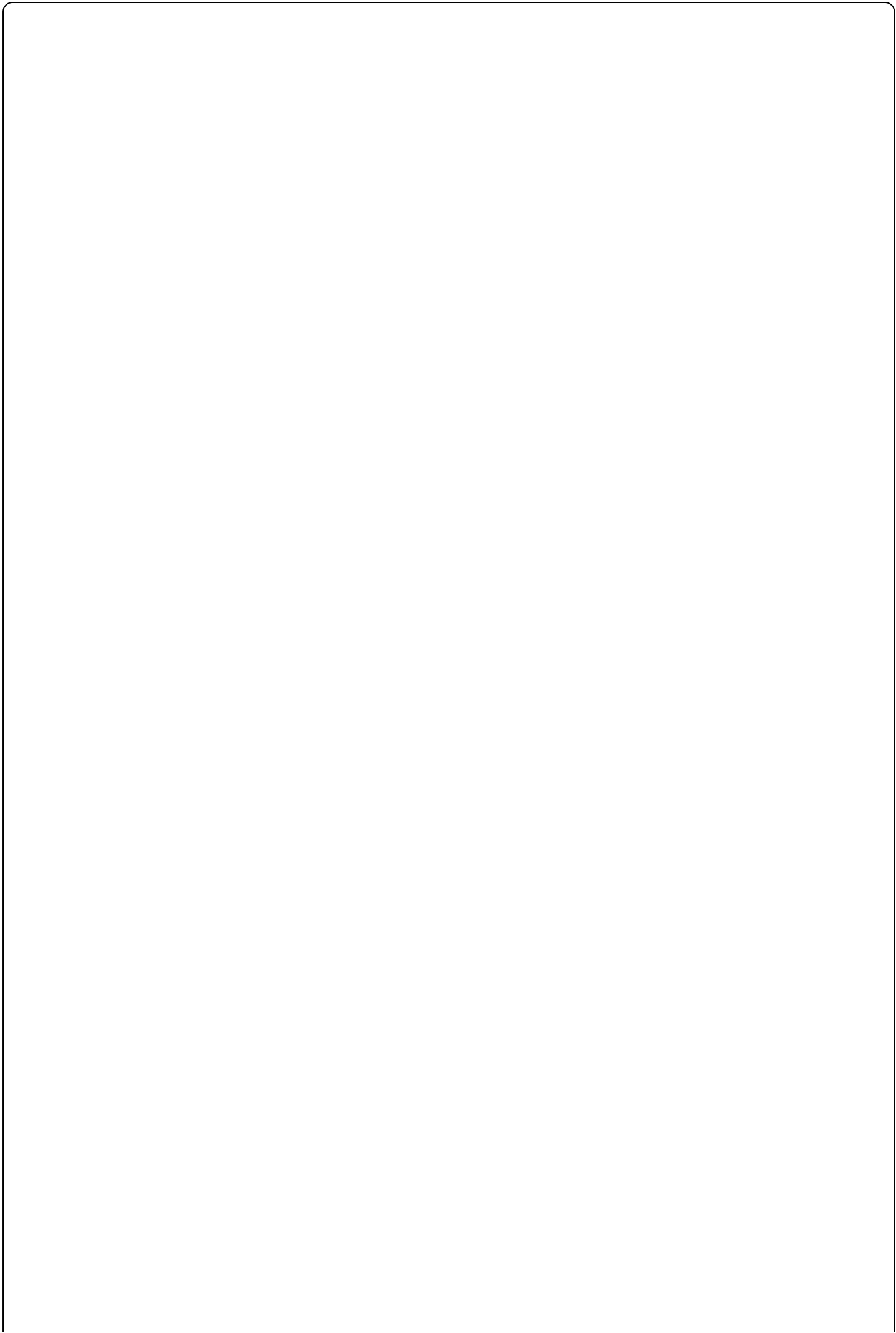
Best for: Applications requiring **schema-validated structured outputs** (JSON APIs, form-filling, data extraction), type-safe LLM calls, and teams wanting a composable validator pipeline. Also good as an output layer alongside other input-focused tools.

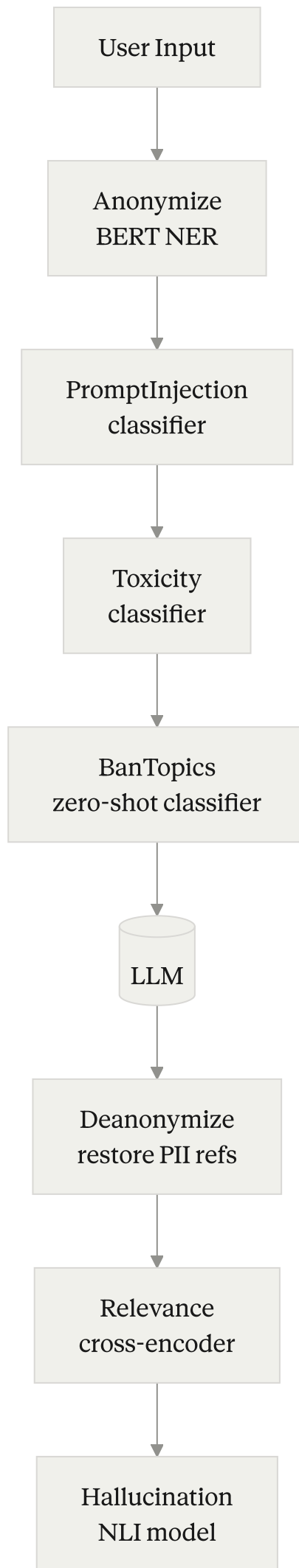
License	Apache 2.0
GitHub	github.com/guardrails-ai/guardrails
Stars	~4.9k
Language	Python, JavaScript
Install	<code>pip install guardrails-ai</code>

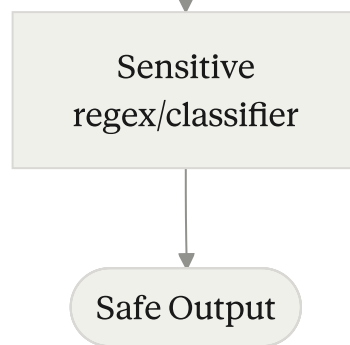
3.3 LLM Guard

What it is: Open-source, **comprehensive input + output scanning library** — often called "the open-source Lakera." Designed as a drop-in security layer for any LLM pipeline with minimal code changes.

How it works: LLM Guard provides two scanner families — `input_scanners` and `output_scanners` — each returning a `(sanitized_output, is_valid, risk_score)` tuple. Scanners chain together; you compose the ones you need. Uses a mix of transformer-based NER (for PII), classifier models (toxicity, prompt injection), and regex patterns.







Input scanners (major ones):

- **Anonymize** — BERT-large NER detects + replaces PII (name, email, phone, SSN, credit card, etc.) with tokens; **Deanonymize** restores in output
- **PromptInjection** — fine-tuned transformer classifier detecting injection patterns
- **Toxicity** — multilingual toxicity classifier (based on Detoxify)
- **BanTopics** — zero-shot classification against forbidden topic list
- **BanSubstrings** — exact keyword / regex blocklist
- **Code** — detect if user is trying to submit code for execution
- **Sentiment** — sentiment bounds enforcement
- **TokenLimit** — prevent context window attacks

Output scanners (major ones):

- **Relevance** — cross-encoder similarity vs. input (catch hallucinated off-topic)
- **Hallucination** — NLI model checks output claims against source passages
- **Sensitive** — regex + classifier for PII, credentials, URLs in output
- **Toxicity** — output toxicity filtering
- **NoRefusal** — detect evasive non-answers
- **Regex** — enforce output format constraints
- **JSON** — validate JSON structure
- **LanguageSame** — ensure response language matches input

Performance: Most scanners run in 10–100ms on CPU; GPU cuts this 3–5×. PII anonymization with BERT adds ~50ms. Much faster than LLM-judge approaches.

Strengths: Comprehensive, modular, no vendor lock-in, strong PII protection, open-source models (no API costs), works fully offline, integrates with Langfuse for observability.

Weaknesses: Requires managing your own model weights/downloads; scanner quality varies (some are ML models that can be fooled); no conversational flow control; community-maintained (slower updates than commercial tools).

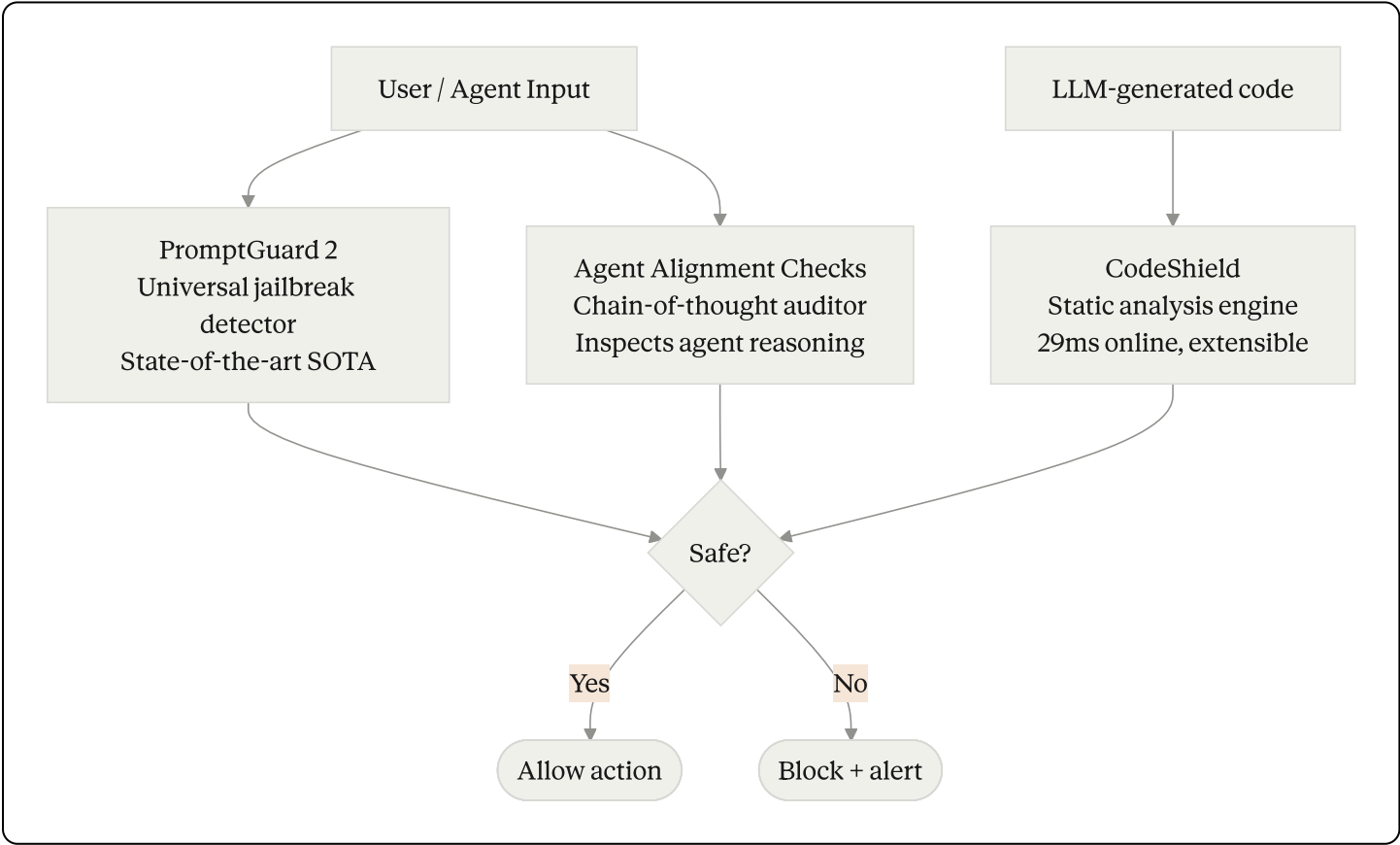
Best for: Teams that need **privacy-first, fully self-hosted, offline-capable** scanning with strong PII detection. Healthcare, finance, on-prem deployments. Also excellent as the OSS alternative to Lakera Guard.

License	MIT
GitHub	github.com/protectai/llm-guard
Stars	~1.3k
Language	Python
Install	<code>pip install llm-guard</code>

3.4 Meta LlamaFirewall (+ LlamaGuard / PromptGuard)

What it is: Meta's **open-source agent-focused security guardrail framework** — released 2025, designed as a "final layer of defense" for AI agents that take real-world actions. Wraps three sub-components: **PromptGuard 2**, **Agent Alignment Checks**, and **CodeShield**.

How it works:



Components:

- **PromptGuard 2:** Universal jailbreak + prompt injection classifier. SOTA performance benchmarks. Detects both direct jailbreaks and indirect injections from untrusted content

(web pages, emails, documents fed into agents).

- **Agent Alignment Checks:** Experimental chain-of-thought auditor that inspects the *agent's reasoning trace* for signs of prompt injection or goal misalignment — stronger than input-only filtering for indirect injections.
- **CodeShield:** Online static analysis engine that runs in 29ms, is fast and extensible, specifically designed to prevent coding agents from generating dangerous or insecure code. Supports custom rule definitions.
- **Customizable scanners:** Regex-based or LLM-prompted scanners any developer can write — low barrier to adding domain-specific rules.

LlamaGuard (v1/v2/v3): Meta's family of fine-tuned safety classification models (7B, 8B, 11B). Uses a taxonomy of harm categories (ML Commons hazard taxonomy); developers can add custom categories. Processes both input and output. Available on HuggingFace, integrated into LlamaFirewall and many third-party tools (DeepEval, NeMo Guardrails).

Strengths: Agent-specific (handles indirect injection in tool use context), SOTA PromptGuard 2, CodeShield for coding agents, fully open-source with Apache 2.0, Meta backing.

Weaknesses: Agent Alignment Checks is still experimental; smaller community ecosystem than NeMo Guardrails; requires managing model weights; no built-in conversational flow control.

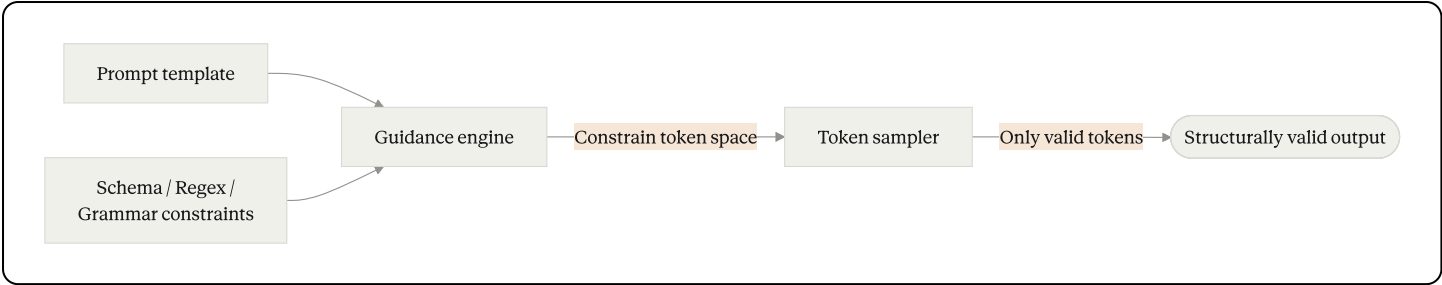
Best for: AI agents taking actions (tool use, code execution, web browsing) where indirect prompt injection via untrusted content is the primary risk. Also excellent for coding agents needing code security analysis.

License	Apache 2.0
GitHub	github.com/meta-llama/PurpleLlama (LlamaFirewall inside)
Models	HuggingFace: meta-llama/LlamaGuard-*
Language	Python

3.5 Microsoft Guidance

What it is: Open-source library (MIT, 19,000+ stars) for **constrained generation** — controlling the LLM's token-by-token output to enforce structure. Instead of filtering *after* generation, it guides the model *during* generation.

How it works: Guidance intercepts the token sampling process and applies constraints — forcing the model to select only from valid tokens at each step. This means JSON schemas, regex patterns, enum membership, or grammar rules can be enforced with **zero post-hoc filtering overhead** and **zero chance of malformed output reaching your application**.



Constraint types:

- `gen()` with regex: force output to match a regex pattern character-by-character
- `select()`: force output to be one of a fixed set of options
- **JSON schema**: enforce full JSON structure with type constraints
- **Grammar**: arbitrary context-free grammar enforcement
- **Interleaving**: mix fixed text with constrained generation regions

Strengths: Strongest structural guarantee of any guardrail approach — malformed output is literally impossible; zero retry overhead; no separate validation call; works with any model that exposes token logits.

Weaknesses: Requires logit access — doesn't work with black-box API-only models (OpenAI API, Claude API via standard endpoints); less useful for semantic / safety guardrails (it only controls structure, not meaning); adds implementation complexity.

Best for: Applications where **output structure must be guaranteed** — JSON extraction, form filling, classification with fixed labels, structured data generation. Essential when you can't afford retry loops and need 100% schema compliance.

License	MIT
GitHub	github.com/guidance-ai/guidance
Stars	~19k+
Language	Python
Install	<code>pip install guidance</code>

3.6 Lakera Guard (now Check Point)

What it is: Enterprise commercial **AI security firewall** — acquired by Check Point in September 2025. Real-time prompt injection, jailbreak, PII, and toxic content detection via a single API call.

How it works: API-based; you pass the prompt to Lakera's endpoint and receive a JSON verdict with risk scores per category. No model weights to manage. SaaS and self-hosted deployment options. Ultra-low latency architecture (<10ms for most checks).

Threat categories: Prompt injection (indirect + direct), jailbreaks, PII exposure, malicious links, off-brand / off-topic, toxic content.

Best for: Enterprises needing **managed, low-latency guardrails with SLA guarantees**, not comfortable managing OSS model weights, and especially those already in the Check Point security stack.

Caveat: Acquisition by Check Point in Sept 2025 means product direction may shift toward Check Point enterprise bundle.

3.7 Giskard (OSS + Red-team / Safety focus)

(Also covered in the Eval guide — Giskard serves dual purpose)

Guardrail capability: Giskard's **LLM Scan** auto-detects 40+ vulnerability types (jailbreaks, prompt injection, PII leakage, stereotypes, hallucinations) at test time. Its **RAGET** module generates adversarial test sets. The OSS evaluation library can be used as a pre-deployment safety gate.

Best for: European teams needing **EU AI Act compliance artifacts**, regulated industries (AXA, BNP Paribas, Michelin users), and teams wanting audit-ready safety scan reports.

License	Apache 2.0
GitHub	github.com/Giskard-AI/giskard-oss
Stars	~5.3k

3.8 ShieldGemma / Gemma Guard (Google)

What it is: Google's family of safety classifiers built on Gemma models. ShieldGemma models (2B/9B) classify content against four harm categories: dangerous content, hate speech, sexually explicit content, and harassment. ShieldGemma-2 (2025) also handles image + text.

Best for: Teams in the Google ecosystem (Vertex AI, Gemini API) wanting a consistent safety layer using Google's own models. Also available on HuggingFace for OSS use.

3.9 AWS Bedrock Guardrails

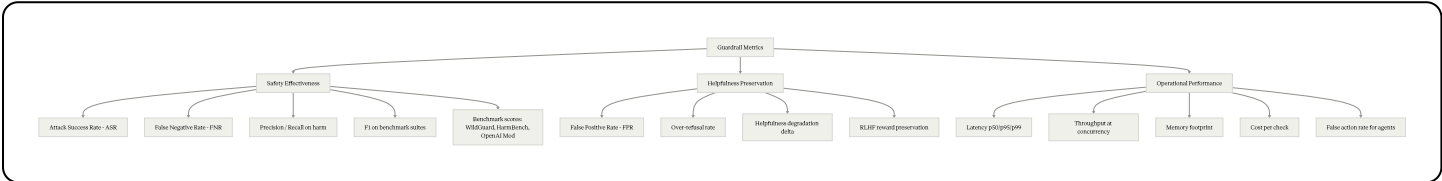
What it is: Managed guardrail service from AWS for applications using Amazon Bedrock. Policy-as-config (no code); configured via AWS Console or API.

Capabilities: Content filtering (hate, insults, violence, sexual content, prompt attacks), PII redaction (20+ entity types), denied topics, grounding check (factual groundedness vs. source), word/regex filters, sensitive information, contextual grounding.

Best for: Teams **already running production workloads on AWS Bedrock** who want zero additional infrastructure and a fully managed safety layer.

4. Metrics Used to Evaluate Guardrails

Guardrail quality is measured across three axes: **safety effectiveness**, **helpfulness preservation**, and **operational performance**.



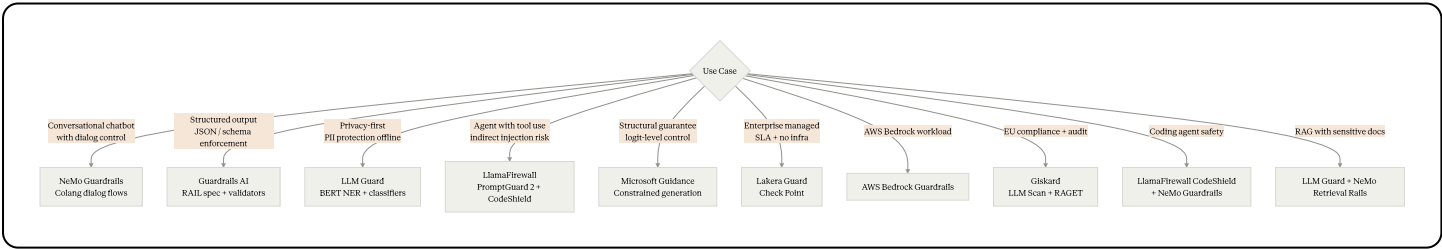
Key metrics explained

Metric	Formula	What it measures
Attack Success Rate (ASR)	Successful attacks / total attack attempts	How often adversaries bypass the guardrail. Lower is better. Primary effectiveness metric.
False Positive Rate (FPR)	Benign inputs blocked / total benign inputs	Over-blocking of legitimate requests. Lower is better. Controls helpfulness cost.
False Negative Rate (FNR)	Harmful inputs passed / total harmful inputs	= 1 – Recall. Missed attacks. Lower is better.
F1 Score	$2 \times P \times R / (P + R)$	Balanced effectiveness. Standard for binary harm classification on benchmark suites.
Precision	True positives / (TP + FP)	Of flagged items, what % were actually harmful
Recall	True positives / (TP + FN)	Of all harmful items, what % were caught
TTFT (Time to First Token)	Time from request to first response byte	Latency impact of guardrail on UX
Guard overhead (µs/ms)	Wall time added by guardrail vs. no-guardrail	Raw latency cost of guardrail layer
Over-refusal rate	Safe completions blocked / total safe completions	The "helpfulness tax" — must be minimized alongside safety
False action rate	Incorrect agent actions allowed or blocked	Agent-specific; an ASR variant for tool calls

Standard benchmark suites for guardrail evaluation

Benchmark	Contents	What it tests
OpenAI Moderation	Diverse harmful content categories	General moderation accuracy
WildGuard	92K human-written + adversarial prompts	Wild-distribution robustness
HarmBench	400 behaviors × 7 attack methods	Systematic adversarial robustness
AdvBench	520 harmful instructions	Standard jailbreak resistance
ToxicChat	Real toxic Chatbot Arena conversations	Real-world toxicity
JailbreakBench	100 behavior × multiple attacks	Reproducible jailbreak benchmark
ALERT	45k red-team prompts across 6 macro-categories	Comprehensive safety scan
MT-Safety	Multi-turn jailbreak sequences	Crescendo-style escalation resistance

5. Use Case → Guardrail Framework Mapping



Detailed mapping by use case

Customer-facing chatbots

Primary risks: Off-topic drift, jailbreaks, brand damage, regulatory violation.

Recommended stack:

- **NeMo Guardrails** for dialog flow + topic control (Colang defines what the bot can and can't discuss)
- **LLM Guard PromptInjection + BanTopics** as fast input pre-filter
- **NemoGuard microservices** (content-safety, jailbreak) for real-time GPU-accelerated classification

RAG-based knowledge systems

Primary risks: PII in retrieved documents surfacing in responses, hallucination, off-source claims, prompt injection via document content.

Recommended stack:

- **NeMo Retrieval Rails** to filter/mask chunks before LLM prompt assembly
- **LLM Guard Anonymize** on retrieved chunks for PII
- **LLM Guard Hallucination** scanner (NLI-based) on output vs. source
- **LlamaFirewall PromptGuard 2** for indirect injection in document content

Metrics to watch: PII leakage rate, hallucination rate (% claims not supported by source), FNR on injection from docs.

Structured data extraction / APIs

Primary risks: Malformed JSON, type violations, schema drift breaking downstream systems.

Recommended stack:

- **Guardrails AI** (RAIL spec) as primary — guarantees schema on output
- **Microsoft Guidance** if logit access available and need zero-retry guarantee
- Input validation: **LLM Guard** or **NeMo input rails** for prompt sanitization

Metrics to watch: Schema compliance rate (target 100%), retry rate (cost proxy), latency per call.

AI agents (tool use, code execution, web browsing)

Primary risks: Indirect prompt injection via web content / emails / files, goal misalignment in reasoning, dangerous code generation, unintended high-impact actions.

Recommended stack:

- **LlamaFirewall** (PromptGuard 2 + Agent Alignment Checks + CodeShield) as primary
- **NeMo Guardrails** with LangGraph integration for flow control
- **Human-in-the-loop approval** for irreversible actions (money, emails, file deletion)
- **Execution rails** to validate tool call inputs before dispatch

Metrics to watch: False action rate, indirect injection success rate, ASR on multi-turn escalation (Crescendo-style), agent task completion rate (guardrails must not over-block legitimate tasks).

Healthcare / Finance / Legal (regulated industries)

Primary risks: PII/PHI leakage, hallucinated medical/legal/financial advice, EU AI Act non-compliance, audit gaps.

Recommended stack:

- **LLM Guard** (offline, all models self-hosted, no data leaving infrastructure)
- **Giskard** for pre-deployment audit artifacts and continuous scanning
- **AWS Bedrock Guardrails** (if on Bedrock) or **Azure AI Content Safety** for cloud-managed baseline
- **NeMo Guardrails** for topic confinement (no off-label medical advice)
- **Custom validators** in Guardrails AI for domain-specific compliance rules

Metrics to watch: PII leakage rate, over-refusal rate (must not block legitimate clinical queries), audit log completeness, EU AI Act documentation artifacts.

Code generation / coding agents

Primary risks: Insecure code (injection vulnerabilities, unsafe functions), unintended file system access, credential exposure.

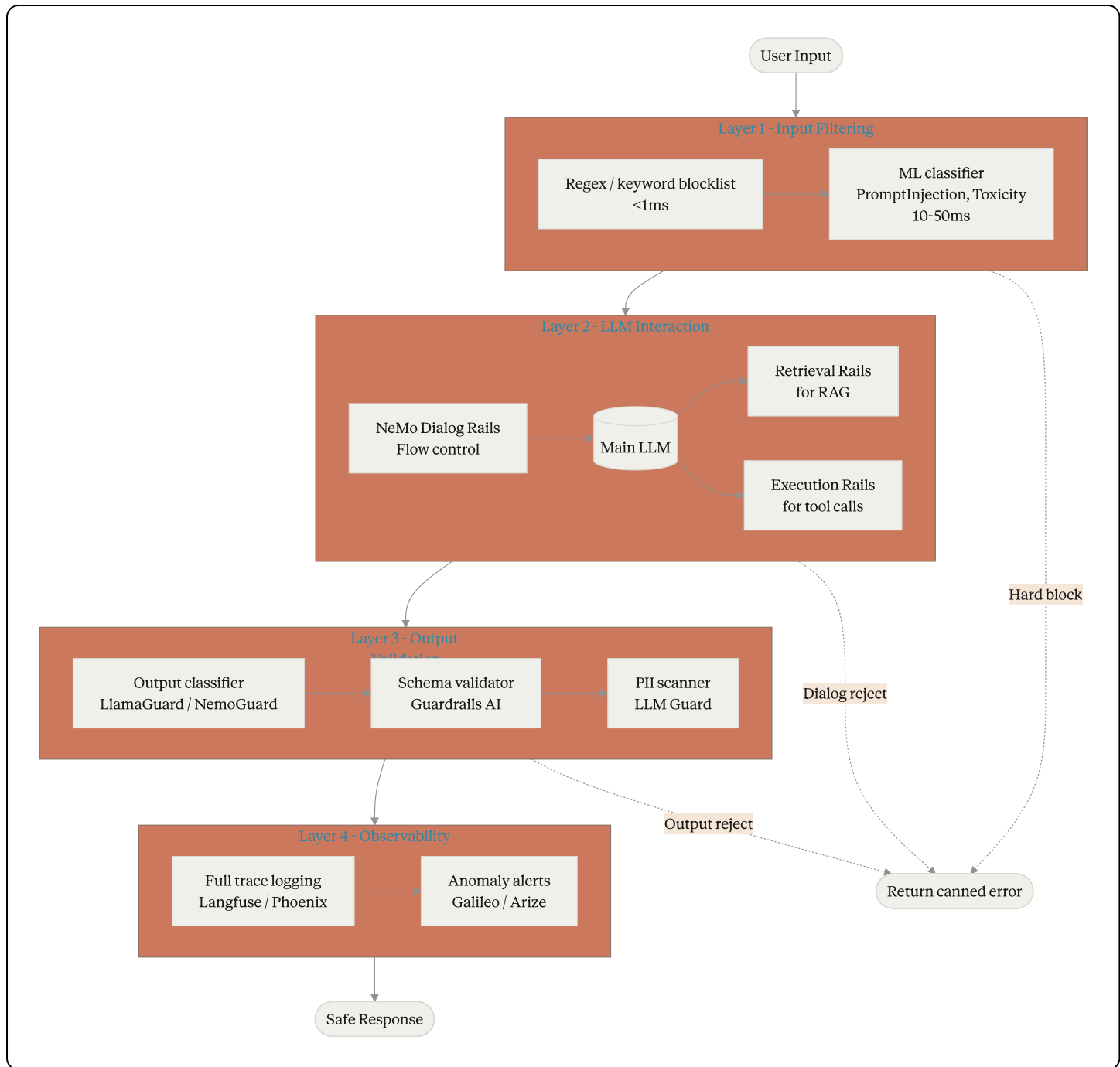
Recommended stack:

- **LlamaFirewall CodeShield** — static analysis in 29ms, extensible rules
- **NeMo Guardrails execution rails** — validate tool call parameters
- **Guardrails AI** with code-specific validators (ban insecure patterns)

Metrics to watch: % generated code passing static analysis, rate of dangerous function calls allowed, false positive rate (overly restrictive blocking of valid code).

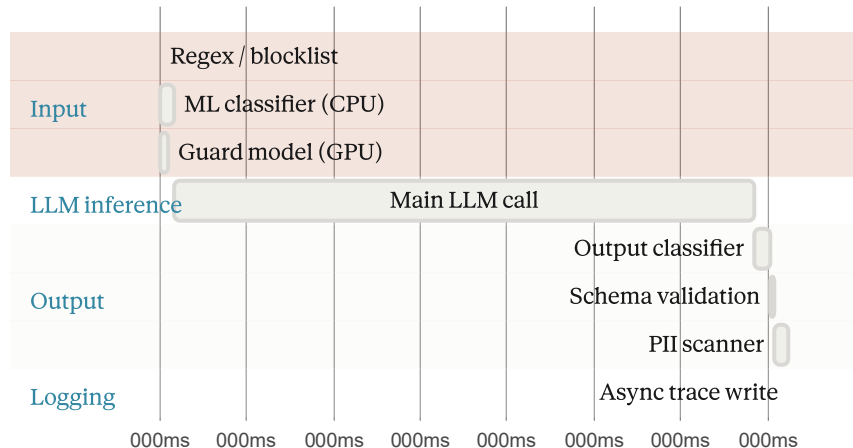
6. Defense in Depth Architecture

No single guardrail is sufficient. Production systems use **layered defense** — multiple independent guardrails at different stages, using different techniques. If one fails, others catch the breach.

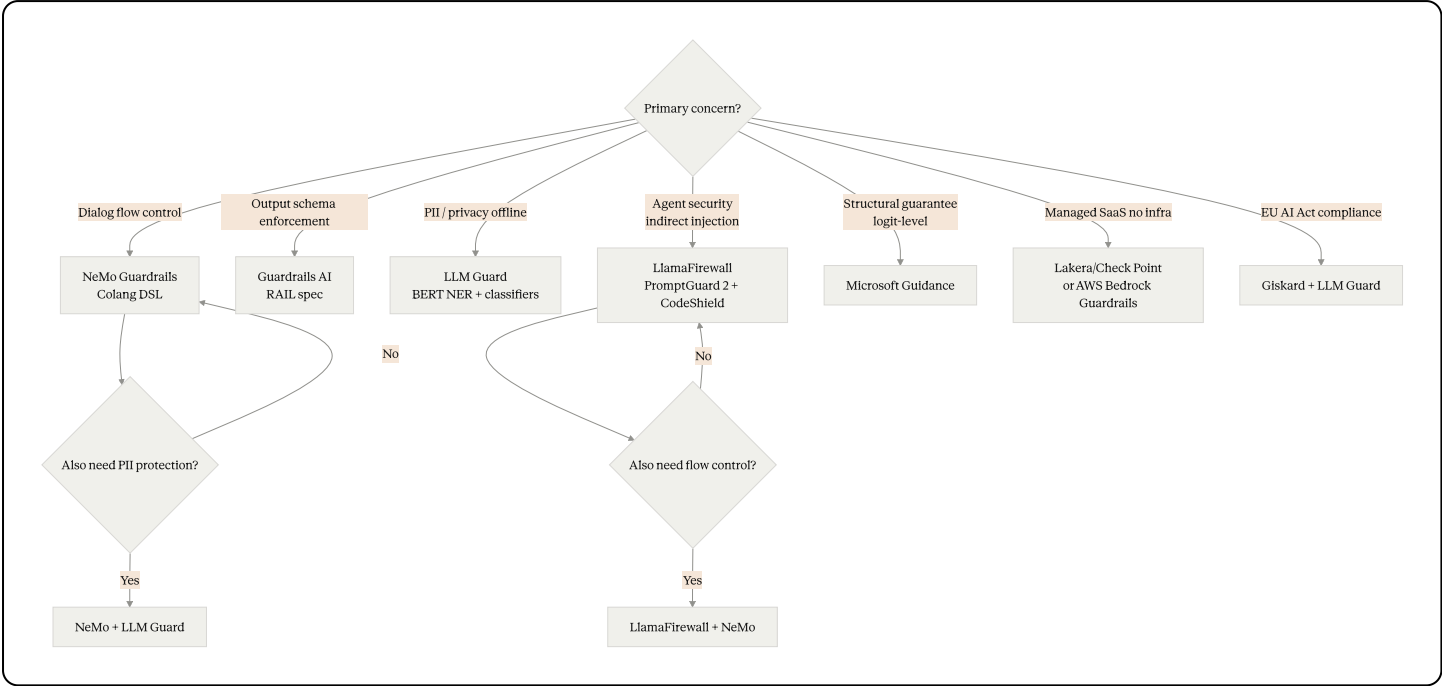


The latency budget framework

Guardrail Latency Budget (Target: <200ms added)



7. Choosing the Right Guardrail



Requirement	Best primary tool	Good complement
Conversational flow control	NeMo Guardrails	LLM Guard for input scanning
Structured output (JSON/schema)	Guardrails AI	Guidance for hard structural guarantee
PII redaction (self-hosted)	LLM Guard Anonymize	NeMo input rails
Prompt injection (managed)	Lakera Guard	LLM Guard PromptInjection
Agent indirect injection	LlamaFirewall PromptGuard 2	NeMo execution rails
Coding agent security	LlamaFirewall CodeShield	Guardrails AI
EU AI Act compliance	Giskard	LLM Guard
AWS ecosystem	Bedrock Guardrails	LLM Guard
Zero latency structural	Microsoft Guidance	N/A (logit-level)

8. Common Pitfalls

Treating guardrails as a magic bullet. No guardrail catches everything. Sophisticated multi-turn jailbreaks (Crescendo escalation), token smuggling, and multi-modal injection can bypass

even the best tools. Guardrails reduce risk — they don't eliminate it.

LLM-judge guardrails in the hot path. A GPT-5 call as safety classifier adds 5-11 seconds. Even GPT-5-mini at 5.6s is untenable for real-time applications. Purpose-built models (LlamaGuard, ShieldGemma, NemoGuard) run in 10-100ms. Use LLM judges asynchronously for audit, not as blocking checks.

Over-refusal destroying the product. A guardrail that blocks 30% of legitimate requests has a severe helpfulness tax. Always measure and minimize **false positive rate** alongside **false negative rate**. The optimal guardrail maximizes F1, not just recall.

Single-layer defense. Depending on a single guardrail — even the best one — is a single point of failure. Run at minimum: a fast input classifier + output validator + async audit log.

Not testing your guardrails. Guardrails degrade as attack techniques evolve. Run adversarial test suites (HarmBench, WildGuard, JailbreakBench, your own domain-specific attacks) on every model or guardrail upgrade. Use DeepTeam, Garak, or PyRIT for this.

Ignoring over-refusal in regulated domains. In healthcare, a guardrail that blocks legitimate clinical queries is a patient safety issue. Carefully tune topic-control policies and always measure domain-appropriate false positive rates.

9. Quick Reference: Framework Comparison

Feature	NeMo Guardrails	Guardrails AI	LLM Guard	LlamaFirewall	MS Guidance
License	Apache 2.0	Apache 2.0	MIT	Apache 2.0	MIT
Input protection	✓	Partial	✓	✓	✗
Output protection	✓	✓	✓	✓	✓ (structural)
Dialog flow control	✓ (Colang)	✗	✗	✗	✗
Agent safety	✓ (LangGraph)	Partial	✗	✓ (primary)	✗
Structured output	✗	✓ (primary)	Partial	✗	✓ (primary)
PII redaction	✓	✓ (via Hub)	✓ (primary)	Partial	✗
Prompt injection	✓	✓ (via Hub)	✓	✓ (SOTA)	✗

Feature	NeMo Guardrails	Guardrails AI	LLM Guard	LlamaFirewall	MS Guidance
RAG retrieval rails	✓	✗	✗	✗	✗
GPU acceleration	✓	✗	✗	✗	✗
Offline / self-hosted	✓	✓	✓	✓	✓
JS/TS SDK	✗	✓	✗	✗	✗
EU AI Act support	Partial	✗	Partial	✗	✗
Stars (GitHub)	~4.5k	~4.9k	~1.3k	Part of PurpleLlama	~19k

10. GitHub Repos & Documentation

Tool	GitHub	Docs
NeMo Guardrails	github.com/NVIDIA-NeMo/Guardrails	docs.nvidia.com/nemo-guardrails
Guardrails AI	github.com/guardrails-ai/guardrails	guardrailsai.com/docs
LLM Guard	github.com/protectai/llm-guard	llm-guard.com
LlamaFirewall	github.com/meta-llama/PurpleLlama	ai.meta.com/research/publications/llamafirewall
LlamaGuard	HuggingFace: meta-llama/Llama-Guard-*	ai.meta.com/research/publications/llama-guard
Microsoft Guidance	github.com/guidance-ai/guidance	guidance-ai.github.io/guidance
ShieldGemma	HuggingFace: google/shieldgemma-*	ai.google.dev
Giskard	github.com/Giskard-AI/giskard-oss	docs.giskard.ai

Tool	GitHub	Docs
Garak	github.com/NVIDIA/garak	docs.garak.ai
PyRIT	github.com/Azure/PyRIT	azure.github.io/PyRIT
DeepTeam	github.com/confident-ai/deepteam	deepeval.com/docs/red-teaming

Conclusion

Guardrails in 2026 are **non-optional production infrastructure**, not optional safety theater. The right architecture:

1. **Layer fast classifiers at input** (regex → ML classifier → guard model in 50–100ms total)
2. **Control conversational flow** with NeMo Guardrails Colang policies for chatbots
3. **Enforce output schemas** with Guardrails AI or Microsoft Guidance for structured data
4. **Protect agents from indirect injection** with LlamaFirewall PromptGuard 2 + Agent Alignment Checks
5. **Protect PII in fully self-hosted setups** with LLM Guard Anonymize
6. **Audit asynchronously** with LLM-judge evaluations in Langfuse or Phoenix
7. **Red-team your guardrails** before every release with DeepTeam, Garak, or PyRIT

The adversarial surface grows every quarter. Your guardrail stack must be tested and updated on the same cadence as your models.